

HW4

Part 1:

Query 1:

I ran a `SELECT *` on the `country_city` view to test it. It worked, showing the country and city info together in one place, which made writing the other queries simpler.

country_code	country_name	city_name	population
US	United States of America	Seattle	750000
US	United States of America	Everett	110000
US	United States of America	Spokane	230000
US	United States of America	Bellevue	150000
US	United States of America	Portland	640000
US	United States of America	Eugene	176000
US	United States of America	Salem	180000
US	United States of America	Gresham	114000
US	United States of America	Portland	68000
US	United States of America	Bangor	32000

(10 rows)

Query 2:

To test the sort order, I set Canada's GDP to be the same as the USA's in `hw3_1.sql`. The query result correctly put Canada first because of its lower inflation, proving the `ORDER BY` clause was working right.

country_code	country_name	gdp	inflation
CA	Canada	65000.00	2.10
US	United States of America	65000.00	2.50
UK	United Kingdom	46000.00	4.20
MX	Mexico	10500.00	6.30

(4 rows)

Query 3:

I tested the border_full view by running a SELECT *. It correctly showed the original borders from hw3_1.sql and also the reverse of each one, like both US-CA and CA-US. The UNION worked as expected.

```
CREATE VIEW border_full AS
SELECT country_code_1, country_code_2, border_length
UNION
SELECT country_code_2, country_code_1, border_length
```

country_code_1	country_code_2	border_length
US	CA	8891.00
MX	US	3145.00
CA	US	8891.00
US	MX	3145.00

(4 rows)

Query 4:

My hw3_1.sql data has the US as a "rich" country bordering Mexico, a "poor" country, based on the query's criteria. The query correctly used the border_full view to find and return the United States.

country_code	country_name
US	United States of America

(1 row)

Query 5:

I added Venezuela and Argentina with the same high inflation of 12.00 to hw3_1.sql to test for ties. The query correctly returned both countries, showing that the NOT EXISTS logic works.

country_code	country_name	inflation
VE	Venezuela	12.00
AR	Argentina	12.00

(2 rows)

Part 2:

```
> psql -U fyoussef -h cps-postgresql.gonzaga.edu -d fyoussef_db
Password for user fyoussef:
psql (16.9, server 14.11 (Ubuntu 14.11-0ubuntu0.22.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

fyoussef_db=> \i pet-data.sql
psql:pet-data.sql:2: NOTICE: table "pet" does not exist, skipping
DROP TABLE
CREATE TABLE
INSERT 0 11
fyoussef_db=> ^Z
[3] + 229554 suspended psql -U fyoussef -h cps-postgresql.gonzaga.edu -d fyoussef_db
> \i pet-data.sql
zsh: i: command not found...
> python sql_test_1.py
5, babe
1, bill
7, donald
11, garfield
3, hobbes
9, jerry
10, odie
6, snoopy
2, spike
8, tom
4, toto
~ > |
```

I faced a few problems while trying to get the python code working. First of which being installing psycopg2 after “sudo dnf install python3-pip” then “pip install psycopg2” which allowed for the library to import and successfully pass this problem.

The second problem I had was related to the name of the config file as part of how the python file was written. It had to be config when originally I had it as config_fy.py, I also initially thought it had something to do with a missing package but that wasn't true.

Another was not being sure which database to connect to till I realized I needed to connect to fyoussef_db as that's the name of mine which needed to be specified in the config file, this fixed the issue.

The last error I had was not running the pet-data.sql file which I did then ran the python code, bringing us to a successful output which is all seen above.